

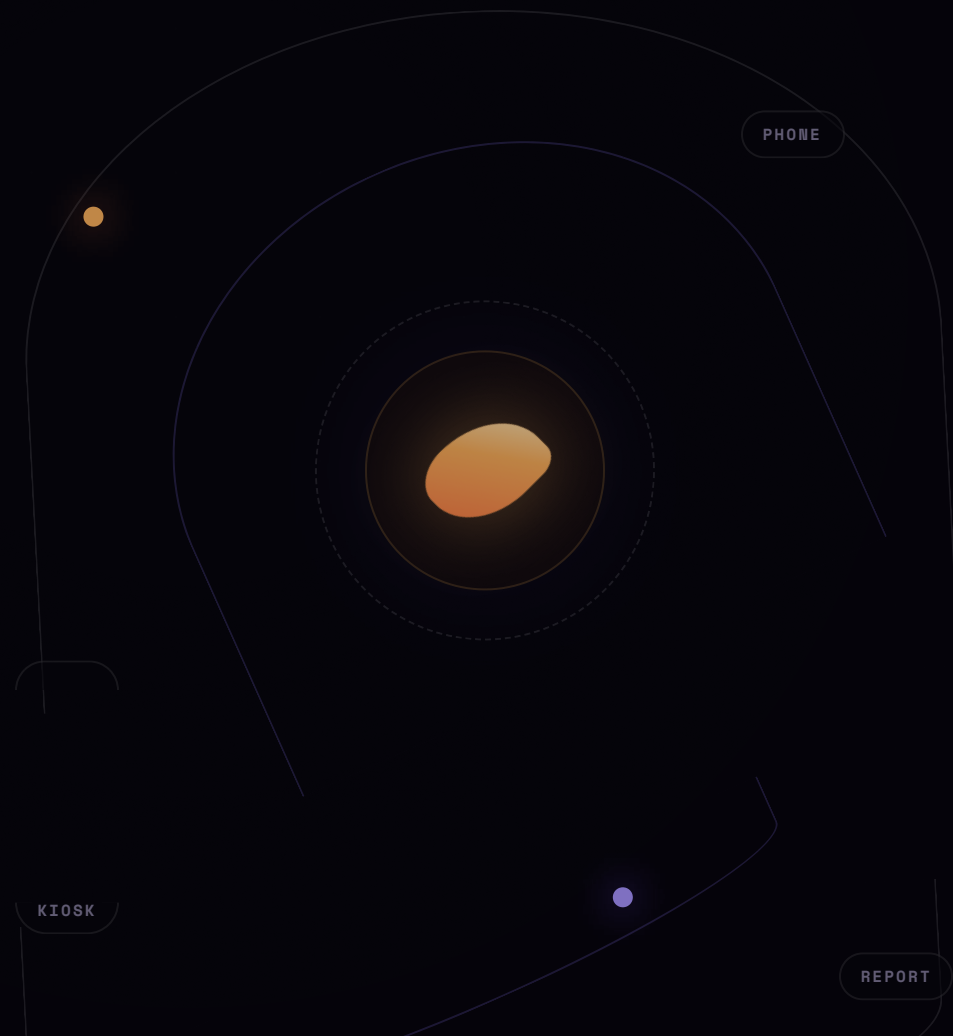
DIYA

A self-running meditation experience where the visitor's phone becomes the kiosk's missing hardware.

IDENTITY

COMPUTER VISION

ZERO-TOUCH FLOW



A visitor walks up. Nobody is there.

The system must identify a stranger, run a camera-guided meditation, deliver a personal report, and reset itself—unattended, all day.

C-01

No staff

No one is present to check visitors in or recover a stalled session.

C-02

No keyboard

A shared input surface creates friction, hygiene, and accessibility problems.

C-03

No scanner

Dedicated reader hardware adds cost, procurement, and another failure point.

C-04

Always on

A kiosk that crashes once and stays down is not an installation.

Two teams. One deliberately thin seam.

HARDWARE TEAM

Cameras · Servos · CV

Depth and thermal capture, posture and gaze analysis, and the external meditation application.

HOME1.py

SHOOT1.py

CHEST1.py

EYE1.py

PROCESS



PDF

MY SCOPE

Everything around it

Kiosk UI, identity flow, backend, web pages, packaging, deployment, and the contract between both sides.

AVALOMIA

EXPRESS

POSTGRES

SYSTEMD

DESIGN DECISION

Launch one process. Read one file. Let both teams evolve independently.

How do you identify someone with nothing to scan them?

Every obvious answer adds hardware: a QR reader, a card reader, or a shared keyboard.



Three attempts. One inversion.

REJECTED

01

Data inside the QR

Offline, but personal data sits on paper and the kiosk still needs a reader.

```
> DIYA1:<base64_json>
```

INCOMPLETE

02

QR as lookup ID

Better privacy. Same physical scanner problem.

```
> GET /api/visitors/:id
```

SHIPPED

03

Invert the interaction

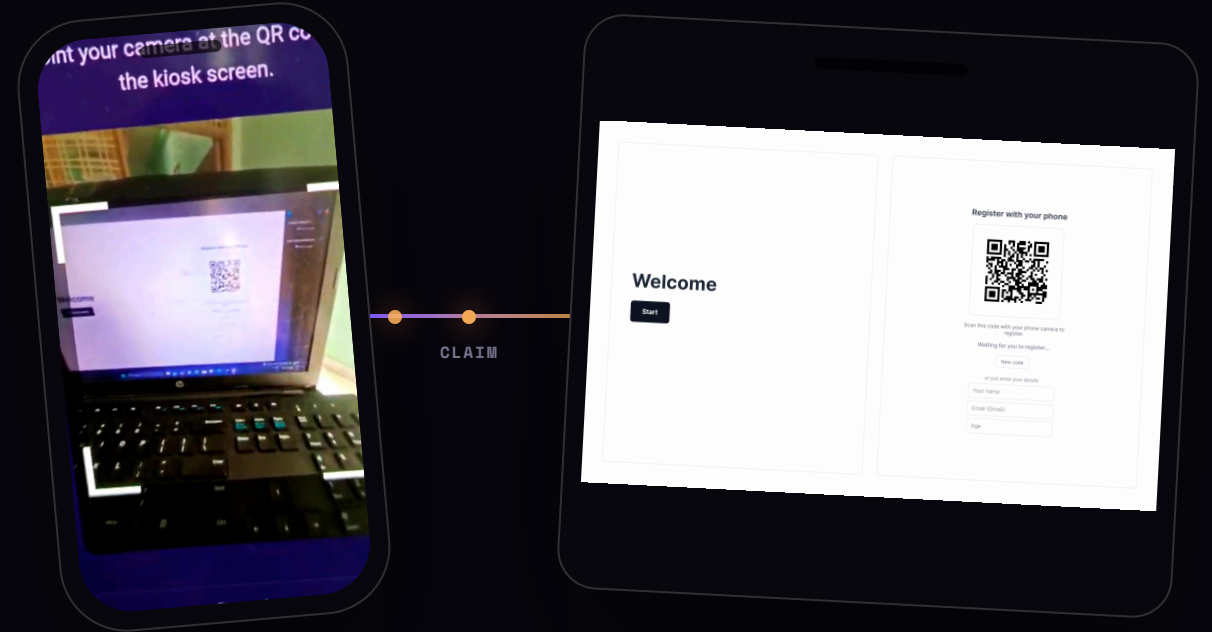
The kiosk shows the code. The visitor's phone scans it. Zero added hardware.

```
> POST /api/claim
```

The phone is the scanner.

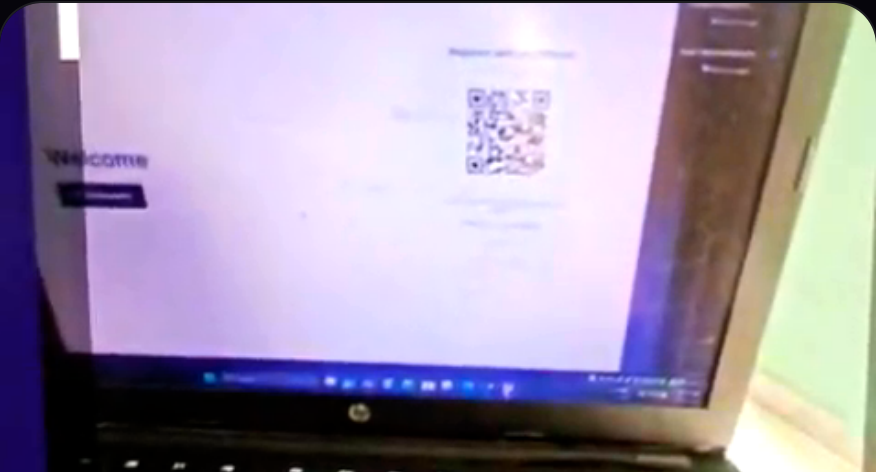
Move the QR from the person to the kiosk. The visitor already carries a camera, keyboard, display, and network connection.

“A constraint produced a simpler design than a budget would have.”

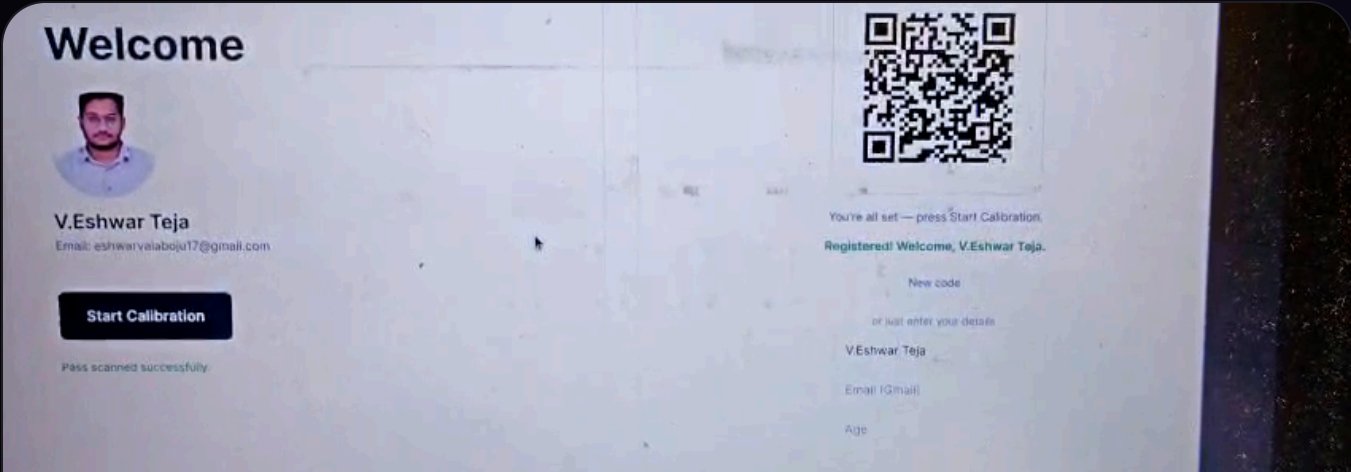


The scan. On real hardware.

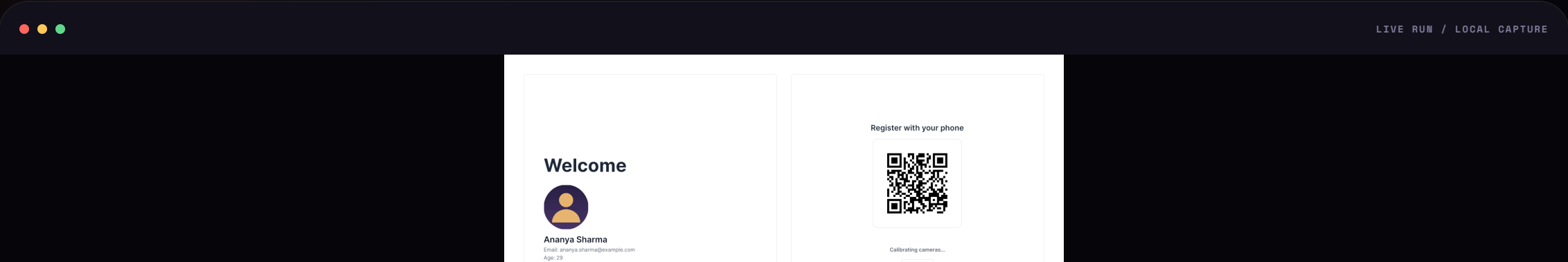
The phone opens a private link, reads the kiosk, and the kiosk advances without a touch.



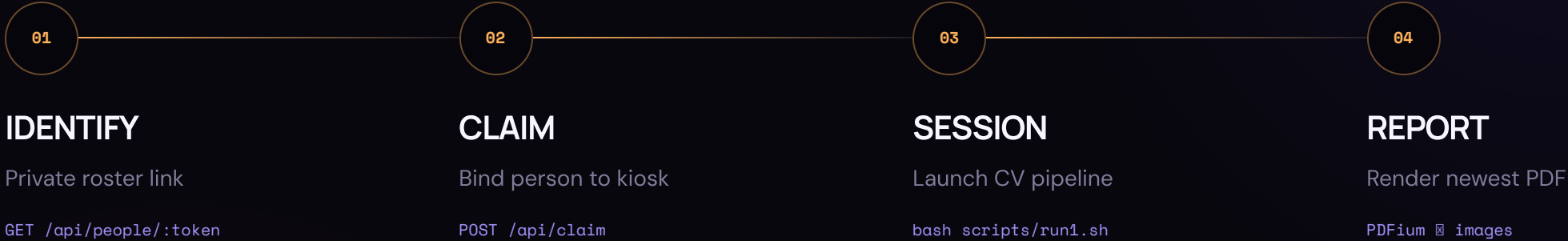
01 PHONE READS KIOSK



02 KIOSK RESPONDS



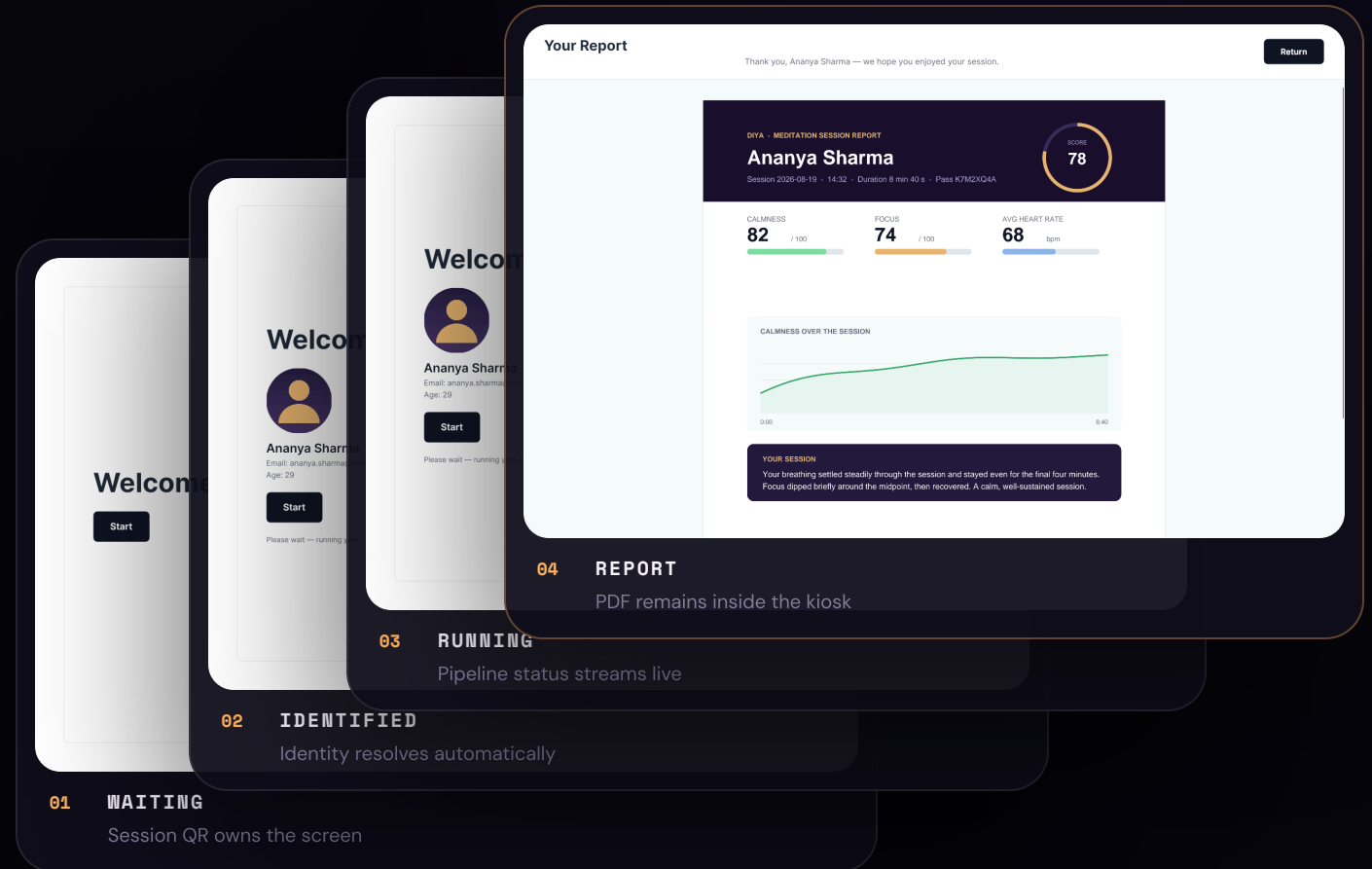
Four stages. One uninterrupted flow.



NETWORK BOUNDARY Phone ↔ Backend ↔ Kiosk

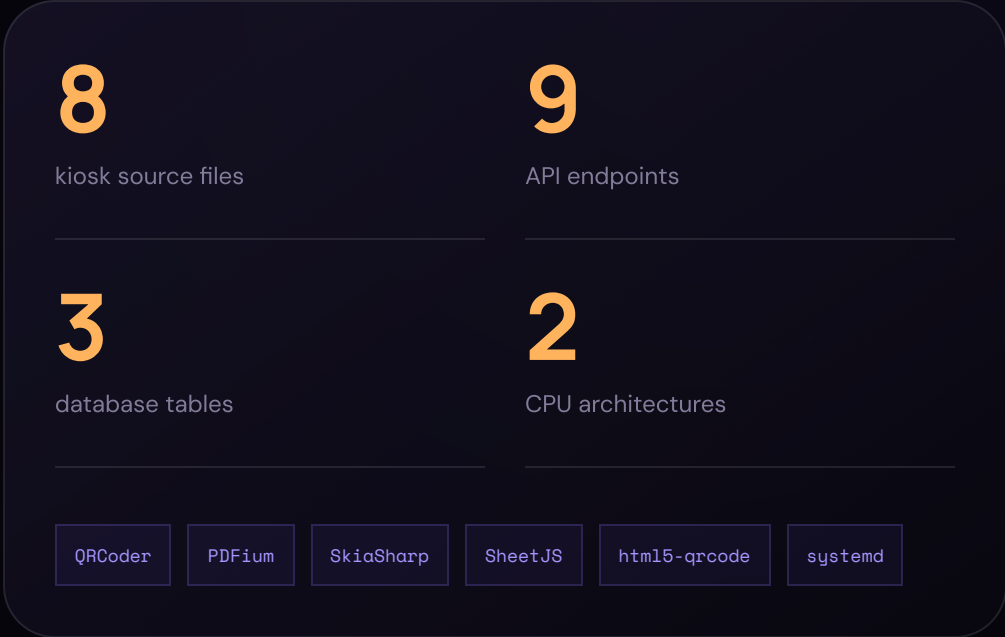
LOCAL BOUNDARY Process + filesystem

Four states. No dead ends.



7 real interfaces across the visitor, kiosk, and operator journey

Three products. One experience.

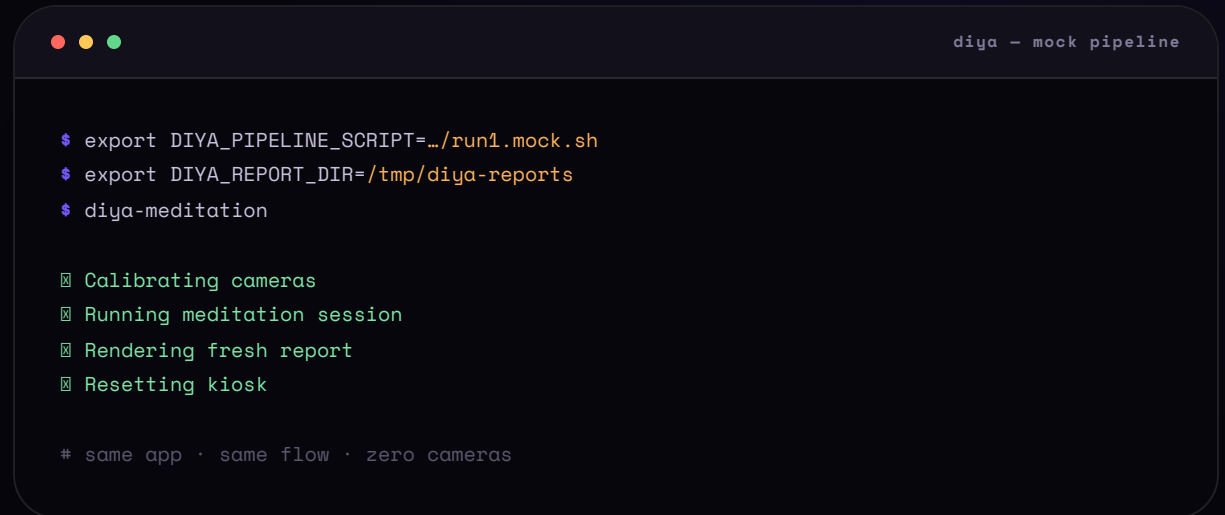


Swap the boundary. Keep the journey.

For most of the build, the cameras and meditation application did not exist. Two environment variables made the entire journey testable.

BUG FOUND

“Newest PDF” could show the previous visitor’s report. The fix accepts only files created after the current session began.



```
$ export DIYA_PIPELINE_SCRIPT=../run1.mock.sh
$ export DIYA_REPORT_DIR=/tmp/diya-reports
$ diya-meditation

☑ Calibrating cameras
☑ Running meditation session
☑ Rendering fresh report
☑ Resetting kiosk

# same app · same flow · zero cameras
```

From repository to unattended machine.

01

BUILD

A single package

```
./deploy/build-deb.sh 1.6.0
```

Self-contained amd64 and arm64 packages. No .NET installation required.

02

LOCK

A kiosk, not an app

```
Restart=always
```

Autostart, crash recovery, fullscreen enforcement, and disabled escape routes.

03

DEPLOY

Backend as blueprint

```
render.yaml
```

Web service and Postgres provision together and track the repository.

CODE

PACKAGE

BOOT

RECOVER

The path works today.

The risks are explicit.

01

PERSONALISE

Pass visitor context into the report pipeline.

02

TIME-BOUND

Add cancellation and a recovery screen for dead cameras.

03

CONSENT

Ask before cameras observe and identity data is retained.

04

INTEGRATE

Connect the production computer-vision hardware.

Walk up. Breathe. Leave with insight.

A constraint became the interaction. The interaction became a system. The system now runs without a guide.

THANK YOU

QUESTIONS?



DIYA